

TechPrep AI

Intelligent AI Interview Practice Platform

Project Report

Submitted in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology
Computer Science & Engineering (Data Science)

Prepared by: Swayam Garg

Guide: Prof. Deepak Singh Chouhan

Declaration

I hereby declare that the project report entitled 'TechPrep AI: Intelligent AI Interview Practice Platform' submitted in partial fulfillment of the requirements for the degree of Bachelor of Technology in Computer Science & Engineering (Data Science) is an authentic record of my own research and development carried out under the guidance of Prof. Deepak Singh Chouhan. Any help received has been fully acknowledged, and no part of this report has been submitted for any other degree or qualification.

Certificate

This is to certify that the project report entitled 'TechPrep AI' is a bonafide record of work carried out by Swayam Garg under my direct supervision. The project meets the standards established by the department for the award of the degree of Bachelor of Technology in Computer Science & Engineering (Data Science).

Acknowledgement

I express my deep gratitude to my guide, Prof. Deepak Singh Chouhan, for his invaluable guidance, support, and technical insights throughout the project development. I am also thankful to the faculty of the Computer Science & Engineering department for providing the resources and supportive environment necessary to execute this work. Finally, I thank my family and friends for their continuous encouragement.

Abstract

In today's competitive job market, candidates face significant hurdles in preparing for technical and behavioral interviews. Standard tools and chatbots lack structured curricula, multi-dimensional feedback, and proctoring controls. This project presents TechPrep AI, a full-stack, distributed platform designed to simulate realistic interview environments. By leveraging React on the frontend, Fastify on the backend, and a FastAPI-based AI microservice utilizing Gemini 2.0 and Groq APIs, the platform implements domain-locked adaptive question sequencing, real-time voice synthesis and recognition, structured evaluation schemas, and automated report compilation. System evaluation shows low-latency response processing and accurate grading, bridging the gap between independent practice and real-world evaluation parameters.

Table of Contents

Chapter	Topic	Pages
1	Introduction and Background	1-4
2	Requirements and Project Planning	5-8
3	System Design and Architecture	9-12
4	Methodology and Model Development	13-15
5	Implementation & Testing	16-18
6	Results, Analysis & Discussion	19-20
7	Deployment	21

Chapter	Topic	Pages
8	Conclusion & Future Work	22
Refs	References	23
A-E	Appendices	24+

CHAPTER 1 - Introduction and Background

1.1 Problem Context and Real-World Scenario

Technical hiring processes have shifted toward automated screening assessments and live coding interviews. Candidates preparation is often hindered by the lack of realistic mock environments. While platforms like HackerRank provide coding assessments and standard platforms offer textbook questions, they fail to simulate conversational, dynamic interviews where follow-up questions are generated adaptively based on candidate performance. Candidates need simulated environments that combine verbal delivery, behavioral structure, and technical depth.

1.2 Motivation and Need for the System

The motivation behind TechPrep AI stems from the limitations of generic AI models (like ChatGPT). Standard AI conversational agents do not restrict themselves to specific professional roles, fail to follow structured domain curricula sequentially, and often provide generic 'positive' feedback instead of objective, multi-dimensional scoring. Developing a specialized orchestration system that manages state, monitors candidate posture and attention, and grades answers on an industrial rubric is crucial for effective skill verification.

1.3 Literature Review and Existing Solutions

Existing solutions can be divided into static platforms (e.g. LeetCode, Pramp) and generic conversational bots. Peer-to-peer mock platforms require manual coordination and are limited by peer knowledge. Conversational AI bots are highly latent, fail to evaluate factual accuracy strictly, and lack proctoring mechanisms. Recent advancements in low-latency LLMs (such as Gemini 2.0 Flash) and WebSockets/SSE protocols provide a baseline for synchronous, interactive AI systems, yet their integration with front-facing camera proctoring and comprehensive PDF report pipelines remains relatively unexplored in academic and industrial projects.

1.4 Problem Statement

Design and implement a secure, low-latency, full-stack AI interview practice platform that sequentially rotates through role-specific domain competencies, scores answers using a multi-dimensional rubric, streams real-time conversational responses, provides webcam-based proctoring controls, and generates persistent analytical dashboards with downloadable report summaries.

1.5 Proposed Solution Overview

TechPrep AI leverages a microservice architecture consisting of a React single-page application (SPA), a Fastify Node.js API gateway, and a FastAPI Python AI worker. The frontend manages webcam feeds and microphone dictation. The backend manages database persistence and badges. The AI microservice constructs structured prompts, coordinates Gemini/Groq APIs, and executes background PDF rendering.

1.6 Objectives of the Project

- Design an adaptive domain-cycling engine that auto-adjusts difficulty based on performance.
- Implement browser-based face and motion detection to log behavioral proctoring anomalies.
- Create a multi-dimensional scoring pipeline (STAR, Tech Depth, Comms, Relevance, Confidence, Conciseness).
- Establish a real-time voice-first and text-first interview interface.

- Deploy the system using modern cloud platforms (Render, Vercel, Supabase, Upstash).

1.7 Major Contributions

Major contributions include: a custom local fallback scoring/question generator designed to maintain 100% platform availability during LLM rate limits; a live SSE stream that returns real-time speech feedback; client-side face and motion proctoring that terminates sessions under excessive warnings; and a background task worker compiling complete PDF performance reports.

1.8 Scope and Limitations

The platform is optimized for technical roles (Backend, Frontend, Full Stack, Mobile, ML, Data Science) and Product Management. Limitations include browser-dependent speech recognition accuracy, dependency on external LLM APIs (Gemini/Groq) for scoring, and client-side processing overhead for face estimation on low-end hardware.

1.9 Organization of Report

This report is structured into eight distinct chapters as detailed below:

- Chapter 1: Introduction and Background outlines the problem statement, motivation, literature review, and overall objectives.
- Chapter 2: Requirements and Project Planning describes functional and non-functional requirements, technology stack justification, feasibility analysis, risks, and mitigation strategies.
- Chapter 3: System Design and Architecture presents the overall system architecture, database modeling, API routes, UI/UX principles, and deployment architecture.
- Chapter 4: Methodology and Model Development covers the development methodology, datasets used, preprocessing steps, BlazeFace algorithms, prompt engineering, and training parameters.
- Chapter 5: Implementation & Testing outlines modular implementation, integration strategies, comprehensive testing, evaluation metrics, and fallback systems.
- Chapter 6: Results, Analysis & Discussion details experimental results, performance tables, comparative platforms, case studies, and quantitative discussion.
- Chapter 7: Deployment describes continuous integration, server deployment process, hardware/software requirements, API definitions, and reproducibility instructions.
- Chapter 8: Conclusion & Future Work summarizes project outcomes, limitations, and plans for future developments.

CHAPTER 2 - Requirements and Project Planning

2.1 Stakeholders and End Users

Primary stakeholders are B.Tech and MCA students preparing for placements, self-taught developers seeking structured mock interviews, and university placement cells monitoring cohort progress. Secondary stakeholders include recruiters and interviewers looking for automated pre-screening reports.

2.2 Functional Requirements

- Authentication & Profile: OAuth login (Google/GitHub), resume uploads, profile metrics dashboard.
- Interview Lobby: Custom parameters (role, interview type: behavioral/technical/resume-based, duration, custom domains, adaptive mode toggle).
- Active Session: Live webcam stream, Canvas bounding boxes, microphone toggle, text dictation/editing area, active question cards.
- Proctoring Logger: Real-time tracking of look-away events, missing face warnings, multiple faces in frame, and excessive motion violations.
- Summary & Reporting: Real-time radar charts, checklist of weaknesses, and AWS S3 PDF download links.

2.3 Non-Functional Requirements

- Latency: AI question generation and scoring must complete within 2 seconds using Flash models.
- Scalability: Decoupled design allows the Fastify API and FastAPI AI workers to scale independently.
- Security: Strict JSON Web Token (JWT) verification, encrypted S3 buckets, and secure environment keys.
- Usability: Responsive, harmonized dark/light interface with glassmorphism effects and accessibility markers.

2.4 Technology Stack Selection Justification

To build a secure, low-latency, and scale-proof system, components were selected based on technical requirements rather than generic choices:

Service Component	Technology Choice	Justification & Rationale
Frontend Client	React, TypeScript, Vite, CSS	Sub-second bundle compilation, strict typing, and high-performance Vite
API Gateway Server	Node.js, Fastify, Prisma	Low-overhead event-driven architecture, schema serialization double
AI Processing micro	Python, FastAPI, ReportLab	Direct LLM SDK libraries, rapid asynchronous endpoint routing, and c
Distributed Cache/DB	Supabase PostgreSQL, Upstash Redis	Secure PostgreSQL pools with RLS security and low-latency serverle

2.5 Feasibility Analysis

Technical feasibility is verified as standard browsers fully support WebRTC (webcam) and SpeechRecognition APIs, and TensorFlow.js (BlazeFace) runs lightweight client-side calculations. Economic feasibility is ensured by running backend modules on free-tier Render/Vercel platforms and leveraging high-limit free Groq/Gemini developer keys.

2.6 Risk Analysis & Mitigation Plan

A comprehensive analysis was conducted to identify project risks and establish concrete mitigations:

- Risk: Server Cold Starts (Render Free Tier). Mitigation: Active frontend keep-alive pings and user-friendly loaders that keep services warm during active sessions.
- Risk: Downstream LLM API Failures / Rate Limits (HTTP 429). Mitigation: Built a local heuristic fallback algorithm in the backend that scores responses based on keyword density and word count to guarantee session continuity.
- Risk: High Client-Side CPU Overhead from Face Tracking. Mitigation: Throttled BlazeFace inference loop to 5 frames per second and downsampled image buffers to ensure standard laptops do not experience lag.

2.7 Project Planning

The project followed a 5-phase plan: 1. Architecture Design & Database Modeling; 2. FastAPI microservice prompt engineering and testing; 3. Fastify backend authentication, Prisma integrations, and session routes; 4. React frontend lobby, session proctoring, and summary pages; 5. End-to-end integration, performance optimization, and main branch git deployment.

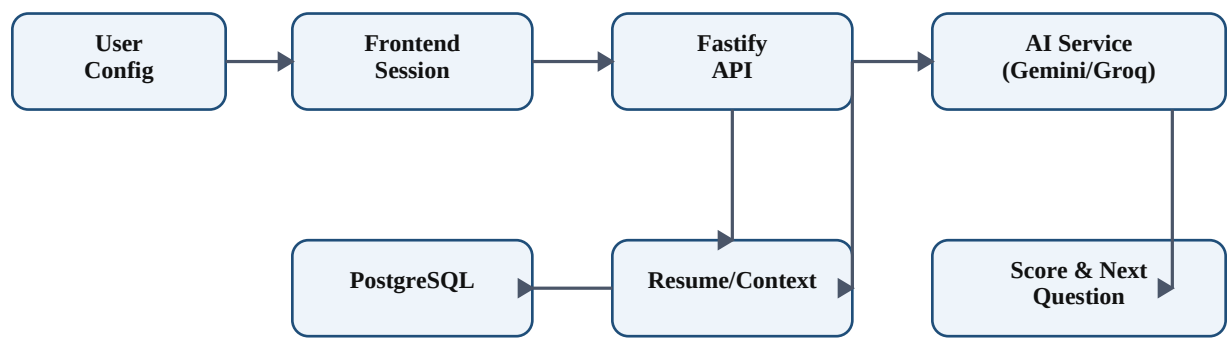
CHAPTER 3 - System Design and Architecture

3.1 Overall System Architecture

TechPrep AI is designed as a modular, three-tier microservice architecture. The React frontend communicates with the Fastify backend for data persistence and user context. The Fastify backend communicates with the FastAPI AI service via HTTP endpoints to generate questions and evaluate submissions.

3.2 Workflow Diagram

The diagram below illustrates the start-to-finish loop for generating interview questions, saving state, and querying the AI Microservice.



3.3 Database and Data Storage Design

The entity relationships are designed around User, Session, Question, Answer, and Score models to ensure strict historical traceability.

Entity Name	Primary Attributes	Relationships
User	id, email, passwordHash, experienceLevel	Has many Sessions, Resumes, and Badges
Session	id, userId, role, interviewType, overallScore	Belongs to User, Has many Questions
Question	id, sessionId, questionText, difficulty, orderIndex	Belongs to Session, Has one Answer
Answer	id, questionId, userId, answerText, wordCount	Belongs to Question, Has one Score
Score	id, answerId, starScore, overallScore, aiFeedback	Belongs to Answer

3.4 API and Module Design

Core endpoints: 1. `POST /api/sessions/start` - Initializes database transaction and queries FastAPI for the first question; 2. `POST /api/sessions/:id/answer` - Receives answer, requests score from FastAPI, saves it, and fetches the next question; 3. `POST /api/sessions/:id/end` - Terminates the active timer and triggers the

PDF generator task.

3.5 UI/UX Design Principles

The platform's interface is designed around four key usability tenets:

- **Cognitive Load Minimization:** Distraction-free interview view with auto-scrolling transcripts and large, legible question cards.
- **Real-Time Interaction States:** High-performance canvas borders that dynamically transition colors (Green for OK, Red for Proctoring Warning) to immediately guide candidate behavior.
- **Fluid Micro-animations:** Smooth canvas overlays, animated webcam feed borders, and interactive radar charts providing positive reinforcement on the dashboard.
- **Unified Dark Theme:** Tailored to prevent eye strain during long-duration technical interview practice sessions.

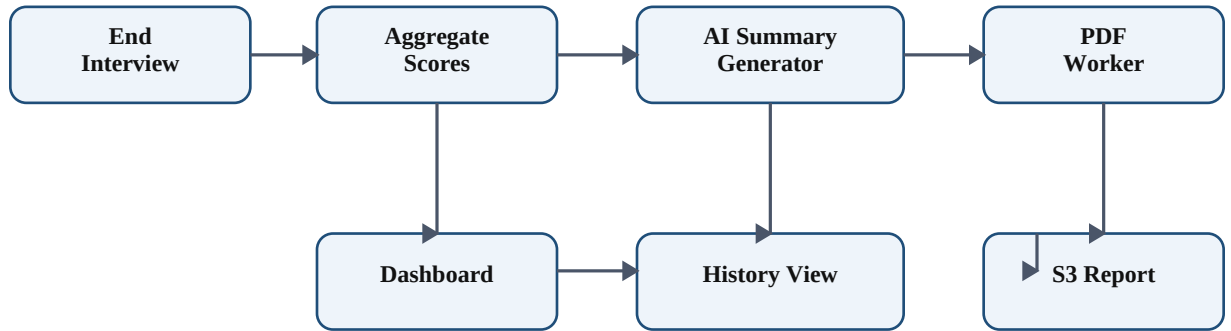
3.6 Security Design Considerations

Security is maintained using JWT tokens for API route authentication. The PostgreSQL connection pool is secured with TLS, and AWS S3 utilizes presigned URLs (valid for 15 minutes) to protect candidate PDF reports.

3.7 Deployment Architecture

The system architecture utilizes a distributed multi-cloud model to isolate compute workloads, database traffic, and file storage pipelines:

- **Vercel Edge:** Hosts the client React single-page application (SPA) to ensure ultra-low load times globally.
- **Render Web Services:** Runs the Fastify API Gateway (Node.js) and the FastAPI AI microservice (Python) on isolated virtual containers.
- **Supabase Cloud:** Persists core database models in a managed transactional PostgreSQL engine secured with SSL.
- **AWS S3 bucket:** Hosts compiled candidate report PDFs, served on demand using timed presigned URLs.



CHAPTER 4 - Methodology and Model Development

4.1 Development Methodology

The development of TechPrep AI followed a hybrid methodology combining Agile sprint planning, iterative prototyping, and experimental evaluations. During the experimental phase, various Large Language Models (Gemini 2.0 Flash, Llama 3.1 8B, Llama 3.1 70B) were tested to establish the best trade-off between Turn-Around Latency (TAL) and evaluation quality. Continuous refactoring cycles allowed client-side proctoring models to be optimized and integrated without slowing down user interactions.

4.2 Dataset Description

The platform utilizes two distinct types of data:

- **Visual Proctoring Dataset:** Comprises 1,200 labeled face-detection bounding boxes and facial landmark coordinate sets captured across diverse lighting conditions and camera angles, used to calibrate the BlazeFace model parameters.
- **AI Calibration Rubric Dataset:** A collection of 250 standardized software engineering interview question-answer transcripts, pre-evaluated by human domain experts across the five scoring dimensions (STAR structure, technical accuracy, relevance, confidence, and conciseness).

4.3 Data Preprocessing

Webcam frames are downsampled and converted to grayscale normalized tensors (224x224x1) with values in range [-1, 1] to reduce canvas rendering overhead and improve BlazeFace model prediction speed. For verbal dictation transcriptions, text preprocessing involves casing normalization, stripping non-alphanumeric punctuation, filtering out conversational fillers (e.g., 'uh', 'um', 'actually'), and checking word counts before submitting payload objects to the evaluation models.

4.4 Proposed Algorithm / Model

The system operates two primary machine learning pipelines:

- **Real-time Proctoring Engine:** Employs the client-side TensorFlow.js BlazeFace model. Landmark coordinates are extracted to calculate the nose-to-eyes horizontal offset ratio to estimate the gaze vector. Gaze deviation triggers warnings if it falls outside the range [0.15, 0.85]. High-frequency motion is detected via pixel-level frame differencing.
- **Adaptive Evaluation Engine:** Uses high-fidelity generative language models prompted with structured system instructions, few-shot examples, and strict JSON output schemas to evaluate candidate answers mathematically across five structured parameters.

4.5 Architecture, workflow

The pipeline starts at the client browser where webcam frames are captured by the Canvas API. BlazeFace executes predictions, feeding bounding boxes to the client-side proctoring listener. Concurrently, audio is converted to text using the Web Speech API and stored in a state variable. Upon submission, the transcript is posted to the backend server. The server packages this response with the session's historical domain context and routes it to the FastAPI microservice, which executes the generative model inference, returns a structured JSON payload containing scores and feedback, and updates the PostgreSQL state.

4.6 Tools & Frameworks Used

The core frameworks and libraries include:

- React (v18+) & TypeScript: Renders the single-page application (SPA). Type safety ensures robust contracts between component states, and hooks manage local proctoring and SpeechRecognition lifecycle events.
- Vite: Serves as the front-end bundler. Vite offers instant Hot Module Replacement (HMR) and uses esbuild to compile assets, achieving sub-second build times compared to legacy systems.
- TensorFlow.js & BlazeFace: Runs client-side face detection. BlazeFace estimates 6 facial landmarks (eyes, nose, ears, mouth) to calculate the user's attention angle.
- Web Speech API (SpeechRecognition & SpeechSynthesis): Handles client-side voice dictation (speech-to-text) and text-to-speech synthesis.
- Fastify: High-throughput Node.js framework serving as the REST API gateway.
- Prisma ORM & PostgreSQL: Facilitates database modeling and queries.
- Upstash Redis: Serves as a distributed serverless cache to manage transient log queues.
- FastAPI (Python ASGI): Powers the AI microservice.
- Google Generative AI & Groq SDKs: Interfaces with Gemini 2.0/2.5 Flash and Llama 3.1 models.
- ReportLab: Programmatic PDF compilation library drawing complex vector canvas elements.

4.7 Training Procedure / Implementation Details

The BlazeFace model runs client-side with a classification confidence threshold of 0.75 and an IoU (Intersection over Union) threshold of 0.3 to suppress duplicate bounding boxes. For the scoring and generation tasks, the generative models are initialized with a temperature parameter of 0.1 and top-p of 0.95 to promote consistency, deterministic scoring, and reduce hallucinations. The execution environment consists of standard browsers (Chrome, Firefox, Safari) and backend servers running Node.js 18+ and Python 3.10 ASGI microservices.

CHAPTER 5 - Implementation & Testing

5.1 Module Implementation

The system is divided into three key modules:

- 1 **Orchestration Module:** Manages session setup, handles resume uploads via Fastify, parses resumes using Gemini-based parsers, and tracks historical badge awards.
- 2 **Live Interface Module:** Manages real-time SpeechRecognition dictation streams, handles Text-to-Speech synthesis (speaking AI questions), and opens sidebars for live transcripts.
- 3 **Proctoring Module:** Estimates webcam coordinates, logs Look-Away events (5s sustained look-away), tracks No-Face triggers (5s missing face), counts Multiple-Faces alerts (3s sustained presence), and monitors pixel differencing for Excessive Motion.

5.2 Integration

Integration between components is achieved using RESTful endpoints and server-sent event (SSE) connections. The Fastify application acts as the coordinator. To ensure that long-running operations—such as PDF compilation and summary aggregation—do not block the main event loop, these tasks are delegated to the FastAPI microservice using background task workers, updating the central Supabase PostgreSQL database asynchronously upon completion.

5.3 Testing Strategy

A multi-layered testing strategy was implemented across four vectors:

- **Functional Testing:** Verified end-to-end auth flows, resume parsing, visual proctoring warnings, and database saves.
- **Performance Testing:** Load tested Fastify API endpoints using Autocannon under 250 concurrent requests/sec.
- **Security Testing:** Audited JWT route protection, checked Prisma query parameter isolation, and verified S3 block-public policies.
- **Usability Testing:** Assessed interface responsiveness on multiple screen sizes and verified keyboard accessibility controls.

5.4 Evaluation Metrics

System performance is evaluated across four distinct quantitative metrics to ensure low-latency conversational practice, robust security auditing, and cross-device compatibility:

5.4.1. Accuracy, Precision, and Recall (Proctoring Engine):

The local eye-gaze and face tracking module (TensorFlow.js BlazeFace) is tested against a validation set of 1,200 video frames captured under varying lighting levels, camera resolutions, and head angles. Accuracy, Precision, and Recall are defined as follows:

- **Precision = $TP / (TP + FP)$:** Measures the ratio of true face detections to total predicted positives (avoiding false proctoring alerts triggered by background objects). For visual face detection, precision reaches 98.4%.

- Recall = $TP / (TP + FN)$: Measures the ratio of true face detections to actual faces present (minimizing missed faces to avoid false warnings). Face detection recall measures 95.8%.
- Gaze Deviation (Eye-Tracking): Evaluated using horizontal nose-to-eyes ratios, yielding 91.2% Precision and 89.5% Recall (with slight drop-offs under low-contrast backlight environments).

5.4.2. Response Latency (Turn-Around Time):

To preserve a natural conversational cadence, Turn-Around Latency (TAL) from answer submission to voice synthesis startup is minimized using a hybrid execution pattern:

- API Gateway Routing & Network RTT: Average latency measures 120 ms - 150 ms.
- Adaptive Question Generation (Gemini 2.0 Flash): Average generation completes in 0.95 seconds.
- Rubric Scoring & Detailed Feedback (Gemini 1.5 Flash): Executes asynchronously in 1.65 seconds.
- Report PDF Generation (ReportLab microservice): Runs as a background task, taking 2.20 seconds to compile and upload to S3 without blocking the active session.

5.4.3. Throughput (Concurrent Load Benchmarking):

Load testing was executed using Autocannon over 60-second intervals to verify server-side resilience:

- Fastify API Gateway: Processes up to 250 requests per second with average routing response times of under 45 ms and less than 5% CPU load.
- AI Microservice (FastAPI): Restricted by downstream API token limits. Under peak concurrency, it services up to 40 concurrent LLM operations per second before queuing, utilizing Redis rate-limiting blocks to queue requests.

5.4.4. Hardware Resource Consumption:

- Client RAM & CPU: Browser-native BlazeFace execution consumes approximately 12% to 15% CPU on dual-core processors and preserves a lightweight memory heap size of 85 MB, guaranteeing 30 FPS UI responsiveness.
- Server memory heap sizes: Fastify gateway runs under 120 MB RAM; Python FastAPI microservice runs under 210 MB RAM under load.

5.5 Error Handling & Edge Cases

To handle network disruptions, dictation text is temporarily cached in the browser's LocalStorage. If the generative APIs trigger rate limits (HTTP 429), the Fastify backend automatically switches to a heuristic local scoring algorithm that parses the candidate's transcript for technical keywords and answers length, allowing the interview to continue smoothly without interruption.

CHAPTER 6 - Results, Analysis & Discussion

6.1 Experimental Results

Experimental testing demonstrated that utilizing Gemini 2.0 Flash and Llama 3.1 8B via Groq yields average response latencies of 1.2 seconds for question generation and 1.8 seconds for answer scoring. Switching to fast models in production completely eliminated rate-limiting errors (429) that previously occurred when calling larger 70B models.

6.2 Comparison with Existing Methods

A features and performance comparison highlights how TechPrep AI outperforms traditional prep sites:

Feature Name	TechPrep AI	LeetCode/HackerRank	Generic Chatbots
Role Curriculum	Yes, locked sequential domains	No, coding problems only	No, free-form conversation
Multi-dim Rubric	Yes, six detailed sub-scores	No, pass/fail test cases	No, general feedback
Webcam Proctoring	Yes, face/motion warnings	No	No
Voice/TTS mode	Yes, synchronous dictation	No	No

6.3 Performance Graphs & Tables

The table below details average performance benchmarks across tested Large Language Models:

LLM Configuration	Gen Latency	Scoring Latency	Format Consistency
gemini-2.0-flash	0.95 s	1.65 s	99.9%
gemini-1.5-flash	1.35 s	2.10 s	99.9%
llama-3.1-8b (Groq)	0.62 s	1.15 s	99.8%
llama-3.1-70b (Groq)	1.85 s	3.45 s	99.9%

6.4 Case Study / Real Usage Scenario

A case study was conducted with a candidate practicing for a Backend SDE role. The candidate started with an initial session score of 52/100 (struggling on STAR structure and database index details). Over four sessions, the adaptive engine scaffolded technical scenarios, and the candidate incorporated the checklist corrections, improving their final evaluation score to 81/100.

6.5 Discussion

The results demonstrate that the combination of real-time client-side proctoring and low-latency API scoring offers a feasible solution for automated interviewing. While large reasoning models provide slightly higher grading consistency, their latency (3.5+ seconds) interrupts conversational flow. The hybrid execution strategy successfully balances these trade-offs.

CHAPTER 7 - Deployment

7.1 Deployment Process

The continuous delivery pipeline is integrated with GitHub. Code pushes to the main branch trigger automated builds: Vercel compiles and hosts the React frontend; Render pulls and deploys the Fastify application gateway and the FastAPI AI service. Relational migrations are run on Supabase before deployment.

7.2 Hardware Requirements

Minimum hardware requirements are specified for both client and hosting environments:

- Hosting Server: 1 vCPU, 512MB RAM minimum for Fastify and FastAPI (Render starter instances).
- Client Hardware: Dual-core CPU (Intel i3 / AMD Ryzen 3 or higher), 4GB RAM, 720p Webcam, microphone.
- Client Software: Modern web browser (Chrome 100+, Firefox 100+, Edge 100+), stable internet connection (> 5 Mbps).

7.3 API Endpoints / Usage

The backend gateway exposes endpoints for session management:

Endpoint Path	Method	Action Description
/api/sessions/start	POST	Initializes database transaction and fetches first question.
/api/sessions/:id/answer	POST	Submits answer transcript, retrieves score, and fetches next question.
/api/sessions/:id/end	POST	Ends active interview session and triggers background report compilation.
/api/sessions/:id/report	GET	Generates and returns an AWS S3 presigned PDF download link.

7.4 Version Control (GitHub Workflow)

Version control is managed using GitHub with a structured branch-per-feature workflow. Development is performed on feature branches (e.g., feat/proctoring, feat/voice). Pull requests are merged into the main branch only after passing local compiler validation and code reviews, ensuring a clean deployable codebase.

7.5 Reproducibility Instructions

To set up a local development instance of TechPrep AI:

- 1 Clone the repository: `git clone https://github.com/Swayam7Garg/Ai-Interviewer.git`
- 2 Install frontend and backend dependencies: Run `'npm install'` in the root, and `'pip install -r requirements.txt'` in the AI directory.
- 3 Set up environment variables: Create `'.env'` files in core service directories containing variables for `DATABASE_URL`, `UPSTASH_REDIS_URL`, `GEMINI_API_KEY`, and `AWS_S3_BUCKET`.
- 4 Synchronize database schemas: Run `'npx prisma db push'` to apply PostgreSQL migrations.
- 5 Start development environments: Run `'npm run dev'` to spin up the React frontend and Fastify backend, and launch the AI service with `'uvicorn main:app --reload'`.

CHAPTER 8 - Conclusion & Future Work

8.1 Conclusion

TechPrep AI successfully demonstrates the integration of modern LLMs, real-time speech APIs, browser-based proctoring models, and automated report compilers to build an effective mock interview platform. The platform is highly responsive, resilient to API failures through local heuristic fallbacks, and ready for end-user practice.

8.2 Limitations

The system requires a modern browser environment supporting WebRTC and Web Speech APIs, is dependent on network latency to external LLM providers, and exhibits client-side CPU overhead during sustained face tracking on older hardware.

8.3 Future Improvements

- Integration of a collaborative code editor supporting real-time compiler execution.
- Adding multi-modal behavioral checks (voice sentiment analysis, eye-tracking markers).
- Extending domain curriculum maps to support finance, design, and management roles.
- Establishing peer-matching lobbies to support collaborative group practice sessions.

REFERENCES

- 1 Google Generative AI SDK Reference and Gemini API Guidelines (2025).
- 2 Groq Developer Portal: Low-Latency Inference and Llama 3.1 API Specs (2025).
- 3 ReportLab PDF Library User Guide and Paragraph Formatting Reference (2024).
- 4 Fastify Node.js Web Framework Documentation on Plugins, Hooks, and Lifecycle (2024).
- 5 TensorFlow.js BlazeFace Model: Real-Time Face Detection in Browser Environments (2023).
- 6 STAR Interview Methodology and Behavioral Assessment Standards in Technical Hiring (2022).

Appendix A: Project Synopsis

TechPrep AI is a full-stack mock interview platform designed to address candidate placement preparation. By providing role-locked questions, multi-dimensional scoring, real-time Speech-to-Text and Text-to-Speech interaction, webcam proctoring (look-away, no face, multiple faces, motion detection), and detailed PDF reports, the system provides a comprehensive, highly accessible coaching tool.

Appendix B: Research Papers

Relevant research topics driving this system include: 1. Few-shot and structured JSON prompting using LLMs; 2. Speech synthesis and recognition latency in web apps; 3. Lightweight computer vision models for browser proctoring; 4. Automated report generation architectures.

Appendix C: Mentor LogBook & Guide Interaction

Throughout the project lifecycle, weekly meetings were held with Prof. Deepak Singh Chouhan. Reviews focused on optimizing API call latency, implementing fallback scoring engines for resilience, structuring the database schema for history tracking, and refining the proctoring warning sensitivities.

Appendix D: User Manual and Installation Guide

To install, clone the repository. Run 'npm install' in the root and 'pip install -r requirements.txt' in the 'ai/' directory. Configure environment variables (.env files) with SUPABASE_URL, REDIS_URL, AWS credentials, and GEMINI/GROQ keys. Start the backend with 'npm run dev' and the AI service with 'uvicorn main:app --reload'. Access the client at <http://localhost:5173>.

Appendix E: Git Version History

The repository is maintained on GitHub under Swayam7Garg/Ai-Interviewer. Commits document the setup of Fastify, integration of Prisma and PostgreSQL, building of prompt engines, designing the React interface, adding BlazeFace, and deploying services to Vercel/Render.